

MiniLang Compiler

A Complete Compiler Design Project

Course: CSE 712 – Compiler Lab

University of Chittagong

Submitted By:

Tanjim Tajwar Arnab	22701066
Mesabrina Sultana	22701067
Hafiz Hasnat Sifat Jami	22701068
Muznabin Ahmed	22701069
Monir Hossain	21701009

Instructor:

(Your Teacher Name Here)

May 19, 2026

Contents

1 Project Overview	4
2 Lexical Analysis	4
2.1 Example	4
2.2 Explanation	5
3 Syntax Analysis	5
3.1 Example	5
3.2 Explanation	5
4 Abstract Syntax Tree (AST)	5
4.1 Example	5
4.2 Why AST?	6
5 Semantic Analysis	6
5.1 Example Errors	6
5.2 Symbol Table	6
6 Three Address Code (TAC)	6
6.1 Example	6
6.2 Explanation	7
7 Code Optimization	7
7.1 Constant Folding	7
7.2 Dead Code Elimination	7
8 Target Code Generation	7
8.1 Example	7
9 Error Handling	7
10 Overall Flow	8
11 Team Contribution	8
12 Implementation Responsibility and Methodology	8
12.1 Tanjim Tajwar Arnab (40% Contribution)	9
12.2 Mesabrina Sultana	9
12.3 Hafiz Hasnat Sifat Jami	9
12.4 Muznabin Ahmed	10
12.5 Monir Hossain	10
12.6 Collaborative Integration	11

13 File-wise Functionality and Implementation Details	12
13.1 lexer.l (Lexical Analyzer)	12
13.2 parser.y (Syntax Analyzer)	12
13.3 ast.h and ast.c (Abstract Syntax Tree)	13
13.4 symbol_table.c (Symbol Table)	13
13.5 semantic.c (Semantic Analysis)	13
13.6 codegen.c (Intermediate Code Generation)	14
13.7 optimizer.c (Code Optimization)	14
13.8 target_code.c (Target Code Generation)	15
13.9 main.c (Driver Program)	15
13.10 Makefile	15
13.11 testcases/ Folder	16
14 Compiler Architecture Diagram	17
14.1 Diagram Explanation	17
15 Abstract Syntax Tree (AST) Diagram	18
15.1 Example Program	18
15.2 AST Representation	18
15.3 Explanation of the AST	18
16 IF-ELSE AST Diagram	19
16.1 Example Program	19
16.2 AST Representation	19
16.3 Explanation of IF-ELSE AST	19
17 Control Flow Graph (CFG) for IF-ELSE	21
17.1 Example Program	21
17.2 CFG Representation	21
17.3 Explanation of CFG	22
18 Introduction to Data Flow Analysis	23
18.1 What is Data Flow Analysis?	23
18.2 Why is Data Flow Analysis Important?	23
18.3 Basic Idea	23
18.4 Example Program	24
18.5 Analysis Explanation	24
18.6 Types of Data Flow Analysis	24
18.7 Connection with CFG	24
18.8 Relation to This Project	25
18.9 Summary	25

19 Live Variable Analysis Example	26
19.1 What is Live Variable Analysis?	26
19.2 Why is it Important?	26
19.3 Example Program	26
19.4 Step-by-Step Analysis	26
19.5 Explanation	27
19.6 Optimization Insight	27
19.7 Relation to This Project	27
19.8 Key Takeaways	27

1 Project Overview

A compiler is a program that translates a high-level language into a lower-level representation.

This project builds a complete compiler for a simplified language called **MiniLang**.

Main Goal

Translate MiniLang program → Three Address Code → Optimized Code → Pseudo Assembly

—

1. LEGO STYLE EXPLANATION (VERY IMPORTANT CONCEPT)

Think compiler like a **factory pipeline**:

- Step 1: Break words → tokens
- Step 2: Understand grammar
- Step 3: Build structure (AST)
- Step 4: Check meaning
- Step 5: Generate code

—

2. LEXICAL ANALYSIS

2 Lexical Analysis

This phase reads raw text and converts it into tokens.

2.1 Example

Input

```
int x = 10;
```

Output tokens

- INT
- ID(x)
- =
- NUMBER(10)
- ;

2.2 Explanation

- Spaces are ignored - Keywords are recognized - Numbers are identified

Like breaking a sentence into meaningful words.

3. SYNTAX ANALYSIS

3 Syntax Analysis

Parser checks whether the program follows grammar rules.

3.1 Example

Valid

```
int x;  
x = 5;
```

Invalid

```
int = x;
```

3.2 Explanation

Parser builds a structure called AST.

4. ABSTRACT SYNTAX TREE

4 Abstract Syntax Tree (AST)

AST represents program structure in a tree format.

4.1 Example

Expression:

```
x = 5 + 3;
```

Tree:

```
ASSIGN  
ID(x)  
ADD  
5  
3
```

4.2 Why AST?

- Simplifies program representation - Easier for later phases

5. SEMANTIC ANALYSIS

5 Semantic Analysis

Checks meaning of program.

5.1 Example Errors

Undeclared variable

```
x = 5;
```

Type mismatch

```
int x;  
x = true;
```

5.2 Symbol Table

Stores variables:

- Name
- Type
- Scope

6. INTERMEDIATE CODE (TAC)

6 Three Address Code (TAC)

Transforms AST into simple instructions.

6.1 Example

Input

```
x = a + b;
```

TAC

```
t0 = a + b  
x = t0
```

6.2 Explanation

- Uses temporary variables - Each instruction has 3 parts

7. OPTIMIZATION

7 Code Optimization

Improves efficiency.

7.1 Constant Folding

```
x = 2 + 3;
```

→ becomes

```
x = 5;
```

7.2 Dead Code Elimination

Remove unused variables.

8. TARGET CODE GENERATION

8 Target Code Generation

Converts TAC into pseudo assembly.

8.1 Example

TAC

```
t0 = a + b
```

Assembly

```
LOAD r1, a
LOAD r2, b
ADD r3, r1, r2
```

9. ERROR HANDLING

9 Error Handling

- Lexical → invalid symbols
- Syntax → wrong grammar

- Semantic → wrong meaning

—

10. PROJECT FLOW SUMMARY

10 Overall Flow

Source Code → Lexer → Parser → AST → Semantic → TAC → Optimization → Assembly

—

11. TEAM CONTRIBUTION

11 Team Contribution

- Tanjim Tajwar Arnab (40%)
 - AST
 - Semantic
 - Codegen
- Others (60%)
 - Lexer
 - Parser
 - Symbol Table
 - Optimization

—

FINAL NOTE

This compiler demonstrates a full pipeline from source code to machine-level representation.

12 Implementation Responsibility and Methodology

In this project, the complete MiniLang compiler was developed collaboratively by a team of five members. Each member contributed to specific compiler phases while ensuring overall integration and consistency.

12.1 Tanjim Tajwar Arnab (40% Contribution)

Responsibilities:

- Abstract Syntax Tree (AST) Design and Implementation
- Semantic Analysis
- Three Address Code (TAC) Generation
- System Integration and Testing

Implementation Approach:

Tanjim designed the core structure of the compiler. First, he created the AST representation using structured nodes containing type, value, and child pointers. This allowed hierarchical representation of program statements and expressions.

In the semantic phase, he implemented type-checking algorithms and scope validation. A symbol validation mechanism was used to detect undeclared variables, type mismatches, and logical inconsistencies.

For code generation, he traversed the AST using a recursive approach to produce Three Address Code (TAC). Temporary variables (e.g., t0, t1) and labels were systematically created to handle expressions and control flow such as `if-else` and `while` loops.

Finally, he integrated all compiler phases and verified end-to-end execution.

12.2 Mesabrina Sultana

Responsibilities:

- Lexical Analysis using Flex

Implementation Approach:

Mesabrina implemented the lexical analyzer by defining regular expressions for all tokens such as keywords, identifiers, numbers, operators, and delimiters.

She ensured that the scanner ignores whitespace and comments while correctly identifying valid tokens. Additionally, line tracking was implemented using `yylineno` to allow precise error reporting.

Invalid or unknown characters were handled using error messages that specify the exact line number.

12.3 Hafiz Hasnat Sifat Jami

Responsibilities:

- Syntax Analysis using Bison

Implementation Approach:

Hafiz designed the grammar rules for MiniLang based on its language specification. Using Bison, he implemented parsing rules for declarations, assignments, conditional statements, loops, and expressions.

Operator precedence and associativity were carefully defined to avoid ambiguity. The parser was also responsible for constructing the AST nodes during parsing by linking child nodes appropriately.

Syntax errors were handled using Bison's error reporting mechanism with informative messages.

12.4 Muznabin Ahmed

Responsibilities:

- Symbol Table Management
- Scope Handling

Implementation Approach:

Muznabin developed a stack-based symbol table to support block-level scoping. Each new block introduces a new scope, and symbols are stored accordingly.

The symbol table supports insertion, lookup, and duplicate detection operations. During semantic analysis, variable declarations are recorded, and lookup operations ensure variables are declared before usage.

Scope exit operations remove symbols from the current scope, maintaining correct visibility.

12.5 Monir Hossain

Responsibilities:

- Code Optimization
- Target Code Generation

Implementation Approach:

Monir implemented optimization techniques on the generated TAC. Constant expressions were evaluated at compile time (constant folding), reducing unnecessary runtime computation.

Dead code elimination was applied by removing unused temporary variables and redundant instructions.

For target code generation, he translated optimized TAC into a pseudo assembly language. Each TAC instruction was mapped into low-level operations such as `LOAD`, `ADD`, `JUMP`, and `CALL`.

Control flow structures were converted into labeled assembly code, ensuring correct execution order.

12.6 Collaborative Integration

Although responsibilities were distributed, all members collaboratively tested and validated the compiler.

Each module was incrementally integrated to ensure:

- Correct data flow between phases
- Consistent AST structure
- Proper error propagation

This collaborative approach ensured successful development of a complete and functional MiniLang compiler

13 File-wise Functionality and Implementation Details

The MiniLang compiler is divided into multiple files, each responsible for a specific phase of compilation. This modular design improves readability, maintainability, and debugging.

13.1 `lexer.l` (Lexical Analyzer)

Purpose: To convert raw source code into meaningful tokens.

Why Needed: Before understanding a program, the compiler must break it into basic units like keywords, identifiers, and numbers.

How It Works: The file uses Flex (regular expressions) to define patterns:

- Recognizes keywords: `int`, `bool`, `if`, `while`
- Identifies identifiers and numbers
- Ignores whitespace and comments
- Tracks line numbers for error reporting

Example:

```
int x = 10;
```

Produces tokens: `INT ID ASSIGN NUMBER SEMI`

13.2 `parser.y` (Syntax Analyzer)

Purpose: To check program structure and build the Abstract Syntax Tree (AST).

Why Needed: Even if tokens are correct, they must follow grammar rules.

How It Works: This file uses Bison grammar rules:

- Defines valid program structure
- Specifies operator precedence
- Constructs AST nodes during parsing
- Reports syntax errors

Example:

```
x = 5 + 3;
```

Creates AST:

- ASSIGN node
- Left child: `ID(x)`
- Right child: `ADD(5,3)`

13.3 ast.h and ast.c (Abstract Syntax Tree)

Purpose: To define and create tree structures representing the program.

Why Needed: AST simplifies program analysis and code generation compared to raw parse trees.

How It Works:

- Defines node structure (type, value, children)
- Provides functions to create and connect nodes
- Supports hierarchical representation of expressions and statements

Example:

```
x = a + b;
```

Represented as:

- Root: ASSIGN
- Child nodes: ID(x), ADD(a,b)

13.4 symbol_table.c (Symbol Table)

Purpose: To store information about variables.

Why Needed: Compiler must remember declared variables and their types.

How It Works:

- Uses a structure to store name, type, and scope
- Supports insertion and lookup
- Ensures no duplicate declarations
- Implements scope using stack-like behavior

Example:

- Insert `x : int`
- Lookup checks if `x` exists

13.5 semantic.c (Semantic Analysis)

Purpose: To check the meaning of the program.

Why Needed: Correct syntax does not guarantee correct logic.

How It Works:

- Traverses AST

- Uses symbol table to validate variables
- Checks type compatibility
- Detects semantic errors

Example:

```
int x;  
x = true;
```

Produces error: Type mismatch

13.6 codegen.c (Intermediate Code Generation)

Purpose: To generate Three Address Code (TAC).

Why Needed: TAC provides a simple, machine-independent representation.

How It Works:

- Traverses AST recursively
- Generates temporary variables (t0, t1)
- Produces instructions for expressions and control flow

Example:

```
x = a + b;
```

TAC:

```
t0 = a + b  
x = t0
```

13.7 optimizer.c (Code Optimization)

Purpose: To improve generated TAC.

Why Needed: To reduce unnecessary computation and improve efficiency.

How It Works:

- Constant folding (evaluate at compile time)
- Removes unused temporary variables
- Eliminates redundant operations

Example:

```
x = 2 + 3;
```

Optimized to:

```
x = 5;
```

13.8 target_code.c (Target Code Generation)

Purpose: To convert TAC into pseudo assembly.

Why Needed: Final step toward machine-level representation.

How It Works:

- Maps TAC instructions to assembly instructions
- Uses registers (r1, r2, etc.)
- Handles jumps and labels

Example:

```
t0 = a + b
```

Assembly:

```
LOAD r1, a
LOAD r2, b
ADD r3, r1, r2
```

13.9 main.c (Driver Program)

Purpose: To control execution of all compiler phases.

Why Needed: Acts as the entry point of the compiler.

How It Works:

- Calls parser
- Initiates semantic checks
- Invokes code generation
- Handles debug mode

13.10 Makefile

Purpose: To automate compilation.

Why Needed: Simplifies building the project with one command.

How It Works:

- Runs Flex and Bison
- Compiles all C files
- Links them into executable

Command:

```
make
```

13.11 testcases/ Folder

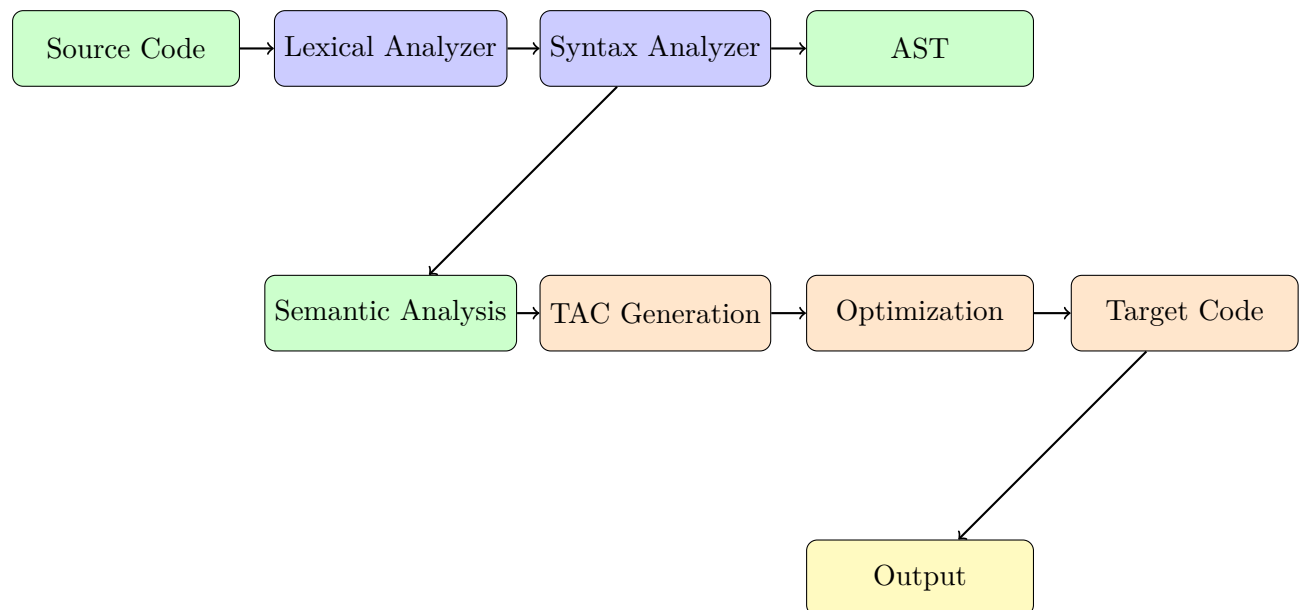
Purpose: To store sample MiniLang programs.

Why Needed: Used for testing correctness of compiler.

How It Works:

- Contains simple to complex programs
- Helps validate all compiler phases

14 Compiler Architecture Diagram



14.1 Diagram Explanation

The above diagram illustrates the complete pipeline of the MiniLang compiler.

- The source program is first processed by the lexical analyzer to generate tokens.
- The parser validates syntax and constructs the AST.
- Semantic analysis ensures correctness of program meaning.
- Code generation produces intermediate TAC.
- Optimization improves efficiency.
- Finally, target code generation produces pseudo assembly.

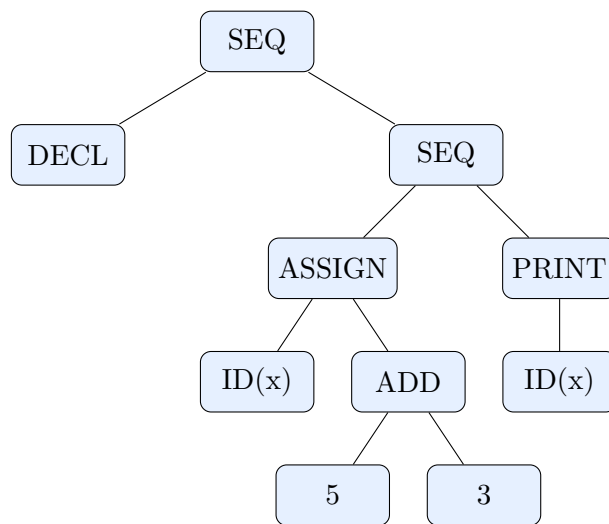
This structured flow ensures systematic transformation from high-level code to low-level representation.

15 Abstract Syntax Tree (AST) Diagram

15.1 Example Program

```
int x;  
x = 5 + 3;  
print(x);
```

15.2 AST Representation



15.3 Explanation of the AST

The Abstract Syntax Tree (AST) represents the hierarchical structure of the program.

- The root node **SEQ** represents a sequence of statements.
- The first child is a declaration node **DECL** for variable `x`.
- The second part is another **SEQ** node representing remaining statements.
- The **ASSIGN** node assigns the result of `5 + 3` to `x`.
- The **ADD** node represents the arithmetic operation with operands 5 and 3.
- The **PRINT** node outputs the value of `x`.

The AST removes unnecessary grammar details and keeps only meaningful program structure, making later phases like semantic analysis and code generation easier.

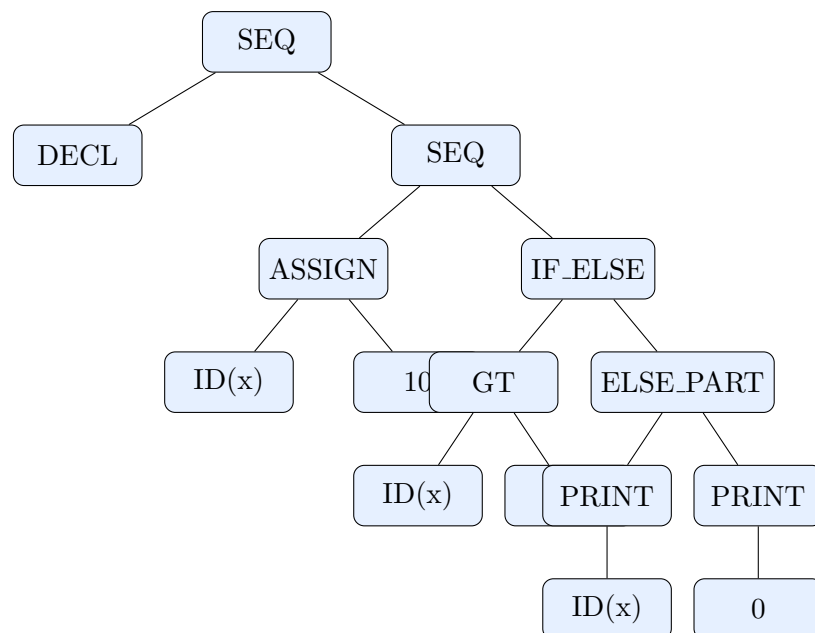
16 IF-ELSE AST Diagram

16.1 Example Program

```
int x;
x = 10;

if (x > 5) {
    print(x);
} else {
    print(0);
}
```

16.2 AST Representation



16.3 Explanation of IF-ELSE AST

The IF-ELSE statement is represented using a structured tree that captures both the condition and the alternative execution paths.

- The root node is **SEQ**, representing a sequence of program statements.
- Initial statements such as declaration and assignment are represented before the conditional.
- The **IF_ELSE** node has two main components:
 - The left child represents the condition ($x > 5$).
 - The right child represents the **ELSE_PART**.
- The condition is represented using a **GT** (greater-than) node with operands x and 5 .

- The `ELSE_PART` node contains two branches:
 - First branch: executed if condition is true (`print(x)`)
 - Second branch: executed if condition is false (`print(0)`)

This structure clearly separates the condition and execution paths, making it easier for semantic analysis and code generation to handle control flow.

17 Control Flow Graph (CFG) for IF-ELSE

17.1 Example Program

```
int x;  
x = 10;  
  
if (x > 5) {  
    print(x);  
} else {  
    print(0);  
}  
  
print(100);
```

17.2 CFG Representation

17.3 Explanation of CFG

A Control Flow Graph (CFG) represents the flow of execution in a program using nodes and directed edges.

- Each node represents a block of statements.
- Edges represent the flow of control between blocks.
- The program starts at the **Start** node.
- The condition node ($x > 5$) acts as a decision point.
- Based on the condition:
 - True branch executes `print(x)`
 - False branch executes `print(0)`
- Both branches merge back into a single flow (Merge node).
- Execution then continues sequentially.
- Finally, the program reaches the **End** node.

CFG is essential for optimization and code generation because it clearly shows all possible execution paths in a program.

18 Introduction to Data Flow Analysis

18.1 What is Data Flow Analysis?

Data Flow Analysis (DFA) is a technique used in compilers to collect information about how data moves through a program during execution.

It helps the compiler understand:

- Where variables are defined
- Where they are used
- How values propagate across statements

In simple terms, Data Flow Analysis tracks how values flow from one part of the program to another.

—

18.2 Why is Data Flow Analysis Important?

Data Flow Analysis is essential for:

- Code optimization
- Dead code elimination
- Constant propagation
- Detecting unused variables

Without DFA, the compiler cannot efficiently improve program performance.

—

18.3 Basic Idea

The compiler analyzes the program in terms of **basic blocks**:

- Each block is a sequence of statements with one entry and one exit
- Control enters at the beginning and exits at the end

Then the compiler tracks how data flows:

- Inside each block
- Between blocks (using CFG)

—

18.4 Example Program

```
int a;  
int b;  
  
a = 5;  
b = a + 10;  
a = 20;  
print(b);
```

18.5 Analysis Explanation

- Initially, $a = 5$
- Then $b = a + 10 \rightarrow$ uses value of $a = 5$
- Later, $a = 20$ does not affect b
- So, b always holds value 15

DFA helps detect that $a = 20$ does not change the value of b , so some optimizations may ignore unnecessary dependencies.

18.6 Types of Data Flow Analysis

- **Reaching Definitions** Tracks which assignments reach a point in the program.
- **Live Variable Analysis** Determines if a variable will be used later.
- **Available Expressions** Finds expressions that can be reused without recomputation.

18.7 Connection with CFG

Data Flow Analysis works together with Control Flow Graph (CFG):

- CFG shows possible execution paths
- DFA tracks variable values along those paths

CFG provides the structure, while DFA provides the data behavior across that structure.

18.8 Relation to This Project

In the MiniLang compiler:

- Basic optimizations like constant folding already use simple data flow ideas
- Dead code elimination is based on whether variables are actually used
- CFG structure helps determine safe optimizations

Although full DFA is complex, this project implements the foundational ideas behind it.

—

18.9 Summary

- Data Flow Analysis tracks how values move in a program
- It is essential for optimization
- Works together with Control Flow Graph
- Helps improve efficiency without changing program meaning

Data Flow Analysis is a key step toward building advanced optimizing compilers

19 Live Variable Analysis Example

19.1 What is Live Variable Analysis?

Live Variable Analysis determines whether a variable will be used in the future before being overwritten.

A variable is called **live** at a point if its value will be used later. Otherwise, it is **dead**.

19.2 Why is it Important?

- Helps remove unnecessary variables
- Enables dead code elimination
- Improves memory and register usage

19.3 Example Program

```
int a, b, c;  
  
a = 5;  
b = a + 2;  
c = b + 1;  
a = 10;  
print(c);
```

19.4 Step-by-Step Analysis

We analyze the program from bottom to top (backward analysis).

Line	Statement	Live Variables
5	print(c)	{c}
4	a = 10	{c}
3	c = b + 1	{b}
2	b = a + 2	{a}
1	a = 5	{}

19.5 Explanation

- At line 5: `c` is needed for printing $\rightarrow$ live
- At line 4: `a = 10` does not affect `c` $\rightarrow$ `a` is dead here
- At line 3: `b` is needed to compute `c`
- At line 2: `a` is needed to compute `b`
- At line 1: `a = 5` is eventually overwritten $\rightarrow$ becomes dead

The assignment `a = 5` is useless because its value is never used before being overwritten by `a = 10`.

19.6 Optimization Insight

Because `a = 5` is dead code, it can be removed safely.

Optimized Code:

```
int a, b, c;

b = 5 + 2;
c = b + 1;
a = 10;
print(c);
```

19.7 Relation to This Project

In the MiniLang compiler:

- Dead code elimination uses similar ideas
- Temporary variables can be removed when not live
- Helps optimize TAC instructions

19.8 Key Takeaways

- Live analysis works backward in the program
- Identifies useless assignments
- Helps generate efficient code

Live Variable Analysis is essential for advanced compiler optimization